# AI ASSISTED LAB ASSIGNMENT – 5.1

Ethical Foundations – Responsible AI Coding Practices

Name: M.Jeshwanth reddy

Roll_no:2503A52392

Batch_no:14

## Task-1: (Privacy in API Usage) Input:

#generate a Python program that connects to a weather API.

```
import os import

requests

def fetch_weather(city: str) -> dict:
    """Return current weather for *city* using
OpenWeatherMap."""
    api_key = os.environ.get("WEATHER_API_KEY")
    if not api_key:
        raise RuntimeError("Set the WEATHER_API_KEY
environment variable first.")

    response = requests.get(
```

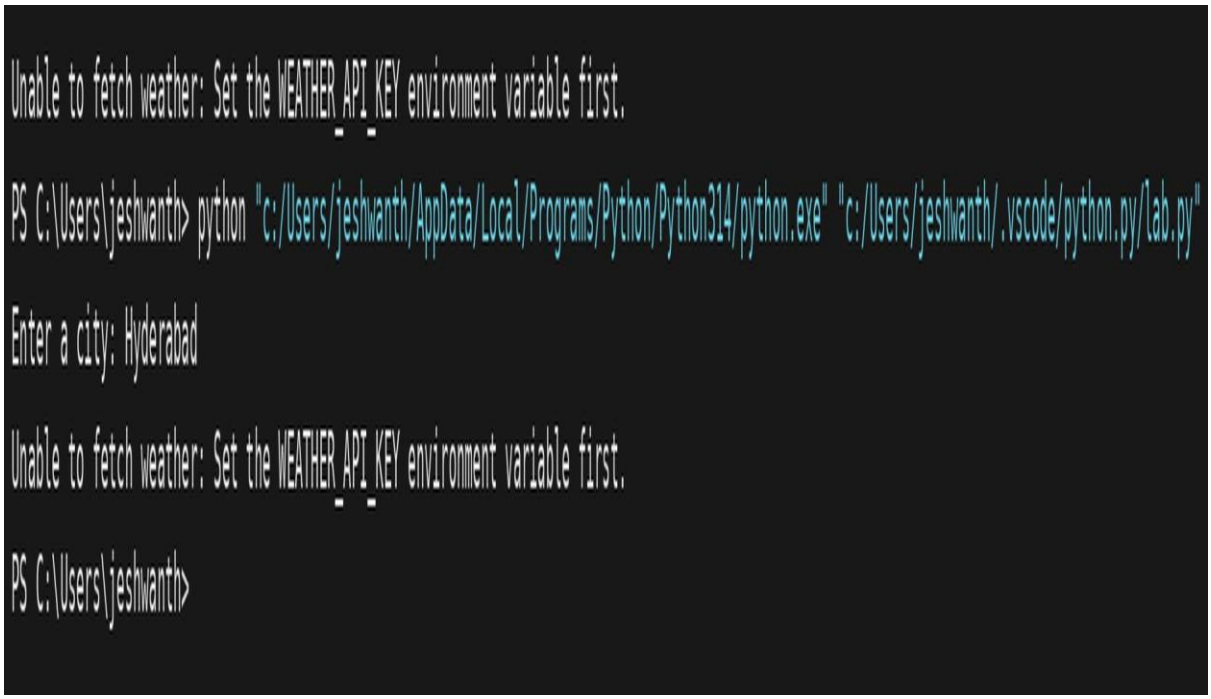
```
        "https://api.openweathermap.org/data/2.5/weather",
        params={"q": city, "appid": api_key, "units": "metric"},
        timeout=10,
    )
    response.raise_for_status()
return response.json()


if __name__ == "__main__":     city =
input("Enter a city: ").strip()     try:
        weather = fetch_weather(city)
        print(
            f"{weather['name']}: {weather['main']['temp']}°C, "
            f"{weather['weather'][0]['description']}"
        )
    except (requests.RequestException, KeyError, RuntimeError)
as error:
        print(f"Unable to fetch weather: {error}")
```

## Explanation:

This task demonstrates how to use a weather API securely without exposing the API key in Python code. An API key is a secret password that allows an application to access an API.

## Output:




# Task-2:(Privacy & Security in File Handling)

## Input:

## #generate a Python script that stores user data (name, email, password) in a file.

```
import base64 import
hashlib import json
import secrets from
pathlib import Path
USER_FILE =
Path("users.json")

def hash_password(password: str) -> tuple[str, str]:
    """Return a unique salt and a slow, one-way password hash."""
    salt = secrets.token_bytes(16)
    password_hash = hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), salt, 600_000
```

```
    )
    return (
        base64.b64encode(salt).decode("ascii"),
        base64.b64encode(password_hash).decode("ascii"),
    )

def save_user(name: str, email: str, password: str) -> None:
    salt, password_hash = hash_password(password)
    user = {
        "name": name,
        "email": email,
        "password_hash": password_hash,
        "password_salt": salt,
    }
    USER_FILE.write_text(json.dumps(user, indent=2), encoding="utf-8")

def main() -> None:
name = input("Name: ")
email = input("Email: ")

    password = input("Password: ")
    save_user(name, email, password)


    print("\nPrivacy analysis:")
    print("- Plain-text passwords would allow anyone with the file to log in.")
    print("- Passwords in logs, backups, or source code could also be exposed.")
    print("- This version stores a salted PBKDF2 hash instead of the password.")
    print(f"\nUser saved securely to {USER_FILE}.")
print(USER_FILE.read_text(encoding="utf-8"))
```

```
if __name__ == "__main__":
main()
```

Explanation:

An insecure AI-generated version might write the password directly:

```
{"name": name, "email": email, "password": password}
```

That creates a serious privacy risk: anyone who reads the file can use the password, and the password may also be exposed through backups or logs.

Passwords should be salted and hashed, not stored in plain text.

Output:

```
PS C:\Users\jeshwanth> reddy\.vscode> & "c:\Users\jeshwanth\reddy\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/jeshwanth/reddy/.vscode/python.py/lab.py"
Name: jeshwanth
Email: reddy@gmail.com
Password: reddy143

Privacy analysis:
- Plain-text passwords would allow anyone with the file to log in.
- Passwords in logs, backups, or source code could also be exposed.
- This version stores a salted PBKDF2 hash instead of the password.

User saved securely to users.json.
{
  "name": "jeshwanth",
  "email": "reddy@gmail.com",
  "password_hash": "qpQjZiy1r9E1mej24XU2cEnLq7Zjc9YQ2b2wIUMjY=",
  "password_salt": "cA1JGN5wBj6CmQ4YuN6Tq+w="
}
```

## Task-3: (Transparency in Algorithm Design)

# Input:

# generate an Armstrong number checking function with comments and explanations. def is_armstrong(number):

"""Return True when number is an Armstrong number; otherwise return False."""

# Armstrong numbers are normally defined for non-negative integers.

if not isinstance(number, int) or number < 0:

return False

# Convert the number to text so that its digits can be examined easily.

digits = str(number)

```
    number_of_digits = len(digits)

    # Add each digit raised to the power of the number of digits.
    digit_power_sum = sum(int(digit) ** number_of_digits for digit in digits)

    # The number is Armstrong when the calculated sum equals the original value.
    return digit_power_sum == number

# Ask the user for a number and remove extra spaces from the input. user_input
= input("Enter a non-negative integer: ").strip()

try:
    # Convert the user's text into an integer.
    value = int(user_input)

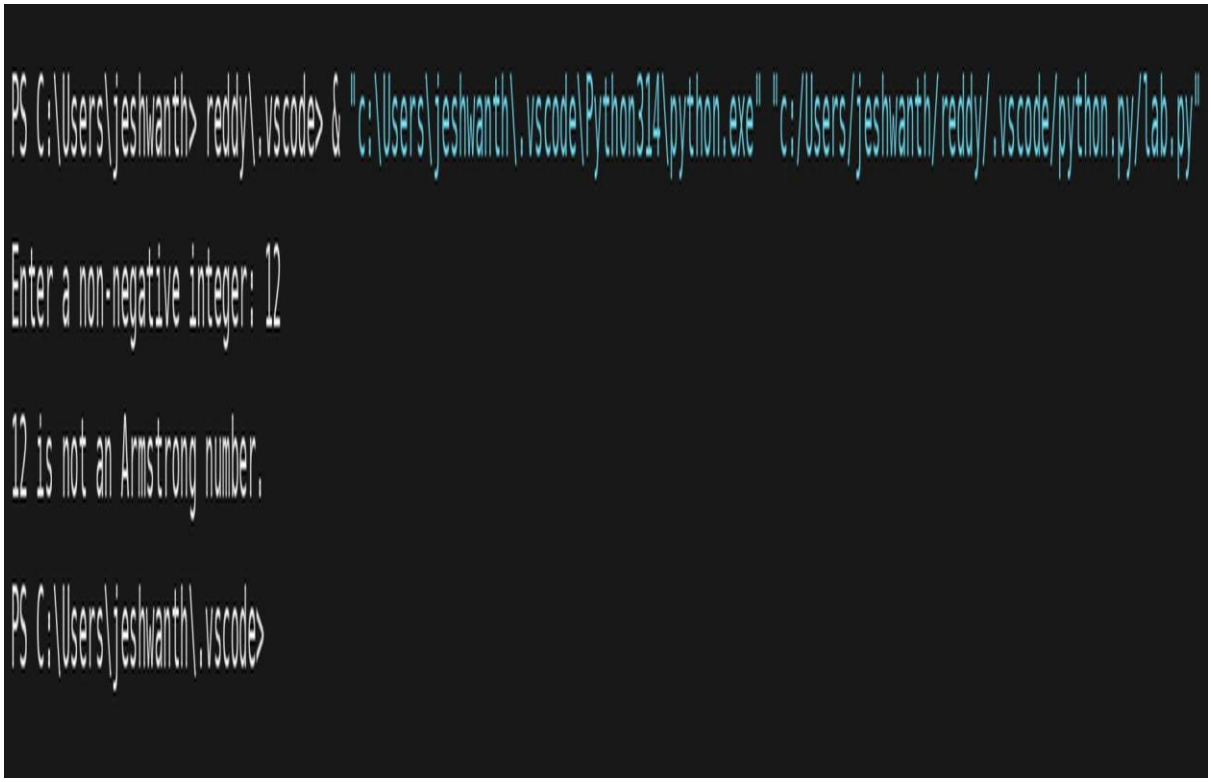
    # Display the result produced by the checking function.
    if is_armstrong(value):
        print(f"{value} is an Armstrong number.")
else:
        print(f"{value} is not an Armstrong number.") except
ValueError:
    # Explain the problem instead of allowing invalid input to crash the program.
    print("Invalid input. Please enter a whole number.")
```

## Explanation:

An Armstrong number is a number equal to the sum of its digits, where each digit is raised to the power of the total number of digits.

For example: 153 = 1**3 + 5**3 + 3**3 = 153.

## Output:




# Task-4: (Transparency in Algorithm Comparison)

## Input:

### Generate Python code for QuickSort and BubbleSort, and include

### comments explaining step-by-step how each works and where they differ

```
def quick_sort(values):
    """Return a sorted copy of *values* using the QuickSort algorithm.

    QuickSort chooses a pivot, divides the remaining values into values that
    are less than or equal to the pivot and values greater than the pivot,
then recursively sorts both partitions.
    """
```

```
    if len(values) <= 1:
        return values.copy()

    pivot = values[len(values) // 2]
    smaller = [value for value in values if value < pivot]
    equal = [value for value in values if value == pivot]
    larger = [value for value in values if value > pivot]

return quick_sort(smaller) + equal + quick_sort(larger)

def bubble_sort(values):
    """Return a sorted copy of *values* using the BubbleSort algorithm.     BubbleSort repeatedly compares neighboring values.  If they are in the     wrong order, it swaps them.  After each pass, the largest unsorted value     has "bubbled" to the end of the list.
    """
    result = values.copy()

    # Each pass checks the portion that has not already been sorted.
    for end in range(len(result) - 1, 0, -1):
        swapped = False

        # Compare neighboring values from left to right.
        for index in range(end):
            if result[index] > result[index + 1]:
                # Swap adjacent values that are out of order.
                result[index], result[index + 1] = (
                    result[index + 1],
```

```
                result[index],
            )
            swapped = True

        # Stop early when a complete pass makes no swaps.
        if not swapped:
break

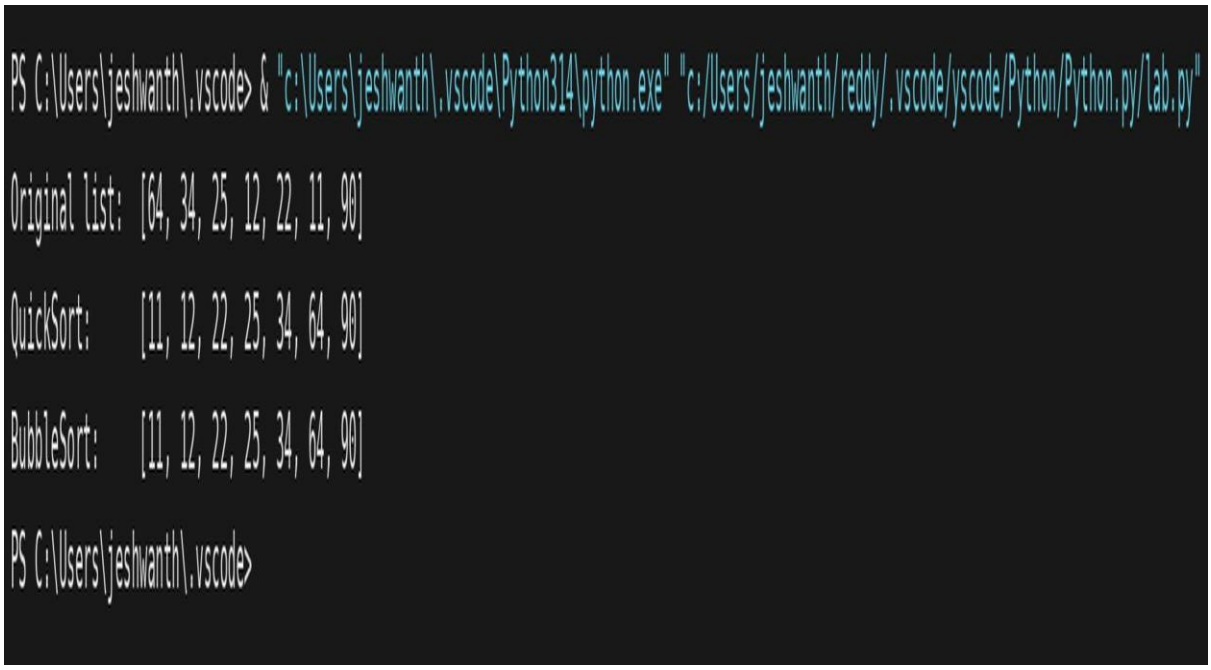
    return result

numbers = [64, 34, 25, 12, 22, 11, 90]
print("Original list: ", numbers) print("QuickSort:
", quick_sort(numbers)) print("BubbleSort:    ",
bubble_sort(numbers))
```

## Explanation:

This program sorts the same list using two sorting algorithms: QuickSort and BubbleSort.

## Output:




## Task-5: (Transparency in AI Recommendations) Input:

### #Generate a recommendation system that also provides reasons for each Suggestion.

"""Product recommendation system with explanations."""

from dataclasses import dataclass from

typing import List, Set, Tuple

@dataclass(frozen=True)

class Product: name:

str

category: str

price: float

```
    features: Set[str]
    description: str


PRODUCTS = [
    Product(
        "SoundMax Wireless Headphones",
        "electronics",
        79.99,
        {"wireless", "noise cancellation", "bluetooth", "long battery"},
        "Comfortable headphones for music, travel, and online classes.",
    ),
    Product(
        "FitTrack Smartwatch",
        "electronics",
        99.99,
          {"fitness tracking", "bluetooth", "water resistant", "long battery"},
        "A smartwatch for tracking exercise, sleep, and daily activity.",
    ),
    Product(
        "ReadPro E-Reader",
        "electronics",
        129.99,
        {"wireless", "long battery", "lightweight", "reading"},
        "A lightweight e-reader with a glare-free display.",
    ),
    Product(
        "ErgoComfort Office Chair",
        "furniture",
```

```
        149.99,
        {"ergonomic", "adjustable", "comfortable", "back support"},
        "An adjustable chair designed for comfortable long work sessions.",
    ),
    Product(
        "UrbanTrail Backpack",
        "bags",
        49.99,
        {"water resistant", "lightweight", "laptop compartment", "travel"},
        "A durable backpack for school, work, and travel.",
    ),
    Product(
        "BrewMate Coffee Maker",
        "kitchen",
        59.99,
        {"automatic", "programmable", "compact", "easy to clean"},
        "A compact programmable coffee maker for home or office use.",
    ),
]

def recommend_products(
    preferences: Set[str],
    budget: float,
    category: str = "",
    number_of_recommendations: int = 3, )
-> List[Tuple[Product, int, List[str]]]:
    """Return products ranked by preference matches, affordability, and category."""
    recommendations = []
```

```
    for product in PRODUCTS:
        if product.price > budget:
continue

        matching_features = preferences.intersection(product.features)
        score = len(matching_features) * 3
        reasons = [f"matches your preference for {feature}" for feature in
sorted(matching_features)]

        if category and product.category.lower() == category.lower():
            score += 2
            reasons.append(f"is in your preferred {category.lower()} category") if not
reasons:
            reasons.append("fits within your budget")

        recommendations.append((product, score, reasons))

    return sorted(
        recommendations,
        key=lambda item: (-item[1], item[0].price),
)[:number_of_recommendations]

def main() -> None:
    print("=== AI Product Recommendation System ===")
    print("Available preferences include: wireless, fitness tracking, ergonomic, travel,
and more.")

    preferences = {
        preference.strip().lower()
        for preference in input("Enter your preferences (comma-separated):
```

```
").split(",")
        if preference.strip()
    }
    budget = float(input("Enter your maximum budget: "))
    category = input("Enter a preferred category (optional): ").strip()

recommendations = recommend_products(preferences, budget, category)

    if not recommendations:
        print("\nNo products match your preferences and budget.")
        return
```

```
    print("\nRecommended products:")
    for position, (product, score, reasons) in enumerate(recommendations, start=1):
        print(f"\n{position}. {product.name} - ${product.price:.2f}")
        print(f"   {product.description}")
        print(f"   Why this is recommended (score {score}):")
for reason in reasons:              print(f"   - {reason}")


if __name__ == "__main__":
main()
```

# Explanation:

This Python program is a simple product recommendation system. It recommends products based on the user's preferences, budget, and category.

# Output:

